

right by the amount stored in `alien_speed`. We track the alien's exact position with the `self.x` attribute, which can hold float values ❶. We then use the value of `self.x` to update the position of the alien's rect ❷.

In the main `while` loop, we already have calls to update the ship and bullet positions. Now we'll add a call to update the position of each alien as well:

alien_invasion.py

```
while True:
    self._check_events()
    self.ship.update()
    self._update_bullets()
    self._update_aliens()
    self._update_screen()
    self.clock.tick(60)
```

We're about to write some code to manage the movement of the fleet, so we create a new method called `_update_aliens()`. We update the aliens' positions after the bullets have been updated, because we'll soon be checking to see whether any bullets hit any aliens.

Where you place this method in the module is not critical. But to keep the code organized, I'll place it just after `_update_bullets()` to match the order of method calls in the `while` loop. Here's the first version of `_update_aliens()`:

alien_invasion.py

```
def _update_aliens(self):
    """Update the positions of all aliens in the fleet."""
    self.aliens.update()
```

We use the `update()` method on the aliens group, which calls each alien's `update()` method. When you run *Alien Invasion* now, you should see the fleet move right and disappear off the side of the screen.

Creating Settings for Fleet Direction

Now we'll create the settings that will make the fleet move down the screen and to the left when it hits the right edge of the screen. Here's how to implement this behavior:

settings.py

```
# Alien settings
self.alien_speed = 1.0
self.fleet_drop_speed = 10
# fleet_direction of 1 represents right; -1 represents left.
self.fleet_direction = 1
```

The setting `fleet_drop_speed` controls how quickly the fleet drops down the screen each time an alien reaches either edge. It's helpful to separate this speed from the aliens' horizontal speed so you can adjust the two speeds independently.

To implement the setting `fleet_direction`, we could use a text value such as 'left' or 'right', but we'd end up with `if-elif` statements testing for the fleet direction. Instead, because we only have two directions to deal with, let's use the values 1 and -1 and switch between them each time the fleet